Module 4: Shor's Algorithm

Quantum Computing Workshop

March 20, 2025

Introduction to Shor's Algorithm

- Developed by Peter Shor in 1994
- Most famous quantum algorithm with practical implications
- Efficiently solves the integer factorization problem
- Exponential speedup over best known classical algorithms
- Threatens RSA and other cryptosystems based on factoring
- Combines quantum and classical processing
- Demonstrates true quantum computational advantage

The Factoring Problem

- Given a composite number N, find its prime factors
- Easy to multiply primes: $N = p \times q$
- Hard to factor composites (reverse the process)
- Best classical algorithm: General Number Field Sieve
  - Runtime: $O(e^{(logN)^{1/3}(loglogN)^{2/3}})$
  - Sub-exponential but super-polynomial
- Shor's algorithm runtime: $O((logN)^3)$ (polynomial)
- This exponential speedup breaks most public-key cryptography

RSA Cryptography

- Most widely used public-key encryption system
- Security based on difficulty of factoring large numbers
- Key generation:
  - Choose two large primes p and q
  - Compute $N = p \times q$ (public)
  - Compute $\phi(N) = (p-1)(q-1)$
  - Choose encryption key e coprime to $\phi(N)$ (public)
  - Compute decryption key d such that $ed \equiv 1 \pmod{\phi(N)}$ (private)
- If factoring is easy, RSA is broken (compute d from N and e)
- Current RSA uses 2048+ bit keys (617+ decimal digits)

Shor's Algorithm: Overview

1. **Problem reduction**: Convert factoring to order finding
2. **Classical pre-processing**: Select random number $a < N$
3. **Quantum subroutine**: Find the order r of a in $\mathbb{Z}_N^*$ (where $a^r \equiv 1 \bmod N$)
4. **Classical post-processing**: Compute factors from order

- The quantum part only solves the order finding sub-problem
- This is where the exponential speedup occurs
- Quantum Fourier Transform is the key to efficient order finding

Order Finding

- For a number a coprime to N, find the order r where:
- $a^r \equiv 1 \pmod{N}$ (smallest such positive r)
- Example: Let $N = 15$ and $a = 7$

$$7^1 \bmod 15 = 7 \tag{1}$$
$$7^2 \bmod 15 = 4 \tag{2}$$
$$7^3 \bmod 15 = 13 \tag{3}$$
$$7^4 \bmod 15 = 1 \tag{4}$$
$$7^5 \bmod 15 = 7 \tag{5}$$
$$\ldots \tag{6}$$

- Here the order is $r = 4$
- Classically, finding this order requires $O(N)$ evaluations
- Quantum approach finds it in $O((\log N)^3)$ operations

Number Theory Background

- Modular arithmetic: a mod N is the remainder when a is divided by N
- Greatest Common Divisor (GCD): Largest number that divides both numbers
- Coprime: Two numbers with GCD $= 1$
- Euler's totient function $\phi(N)$: Number of integers less than N that are coprime to N
- For prime p, $\phi(p) = p - 1$
- For $N = pq$ (product of two primes), $\phi(N) = (p-1)(q-1)$
- Euler's theorem: If $\gcd(a, N) = 1$, then $a^{\phi(N)} \equiv 1 \pmod{N}$

From Order to Factors

If we find order r of a in $\mathbb{Z}_N^*$:

- If r is even, compute $a^{r/2} \bmod N$
- If $a^{r/2} \neq -1 \pmod{N}$, then:

$$a^r \equiv 1 \pmod{N} \tag{7}$$

$$(a^{r/2})^2 \equiv 1 \pmod{N} \tag{8}$$

$$(a^{r/2})^2 - 1 \equiv 0 \pmod{N} \tag{9}$$

$$(a^{r/2} - 1)(a^{r/2} + 1) \equiv 0 \pmod{N} \tag{10}$$

- At least one of $\gcd(a^{r/2} - 1, N)$ or $\gcd(a^{r/2} + 1, N)$ is a non-trivial factor of N

Example: For $N = 15$, $a = 7$, $r = 4$

$$7^{r/2} = 7^2 = 4 \bmod 15 \tag{11}$$

$$\gcd(4 - 1, 15) = \gcd(3, 15) = 3 \tag{12}$$

$$\gcd(4 + 1, 15) = \gcd(5, 15) = 5 \tag{13}$$

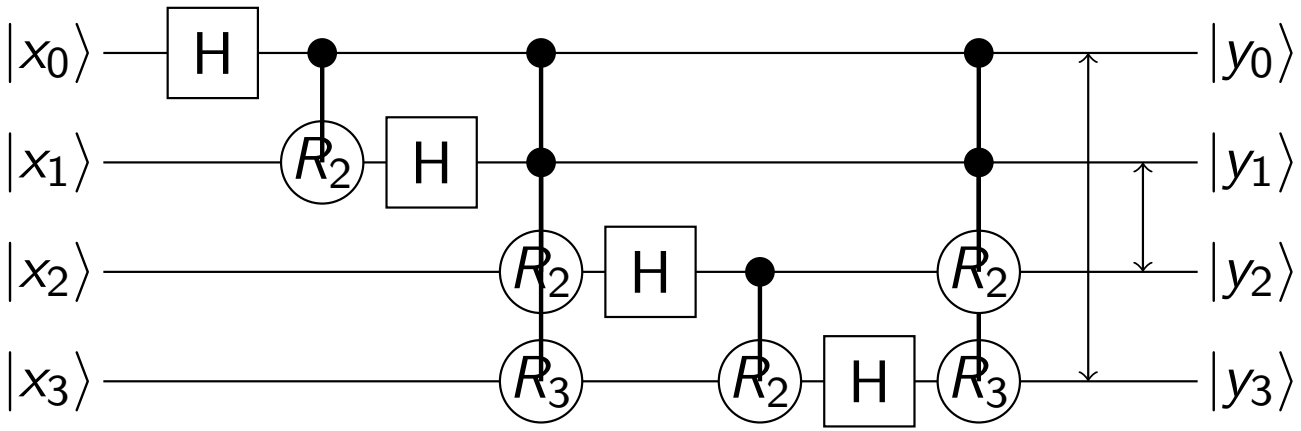
Quantum Fourier Transform

- Quantum analog of the Discrete Fourier Transform
- Maps from position basis to frequency basis
- Action on basis states:

$$QFT_N |j\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi ijk/N} |k\rangle \tag{14}$$

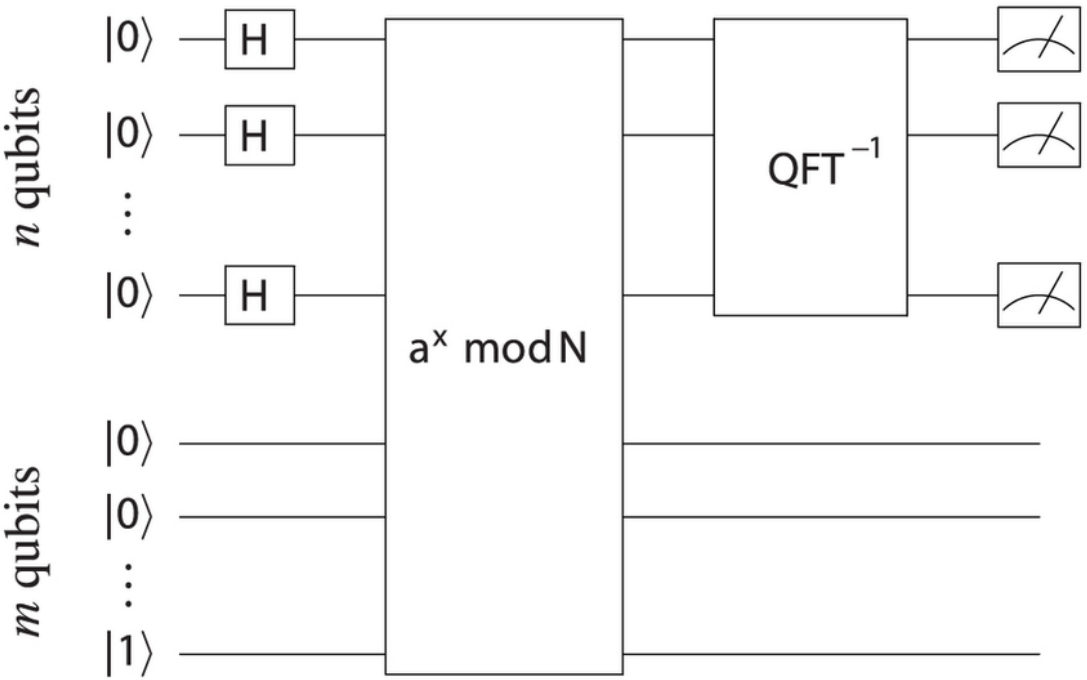
- Efficiently implementable on quantum computers
- Requires only $O((\log N)^2)$ quantum gates
- Classical FFT requires $O(N \log N)$ operations
- Crucial component for order finding and many other quantum algorithms

QFT Circuit



- R_k = controlled phase rotation by $2\pi/2^k$
- Final SWAP gates reverse the order of qubits
- Circuit depth grows quadratically with number of qubits

Quantum Order Finding Circuit



- First register: Encodes superposition of all possible x values
- Second register: Stores function values $a^x \bmod N$
- U_a is the modular exponentiation oracle
- Inverse QFT transforms the first register
- Measurement gives an approximation of s/r (fraction with denominator r)

Shor's Algorithm: Full Procedure

1. **Classical**: Check if N is even, a prime power, or divisible by small primes
2. **Classical**: Choose random a where $1 < a < N$
3. **Classical**: Calculate $\gcd(a, N)$. If $\gcd(a, N) > 1$, we found a factor
4. **Quantum**: Use order-finding circuit to find order r of a in $\mathbb{Z}_N^*$
5. **Classical**: If r is odd, go back to step 2
6. **Classical**: If $a^{r/2} \equiv -1 \pmod{N}$, go back to step 2
7. **Classical**: Compute $\gcd(a^{r/2} \pm 1, N)$ to find non-trivial factors

- Only step 4 requires a quantum computer
- Success probability is $O(1/\log\log N)$ per iteration

Simple Example: Factoring N = 15

1. Choose $a = 7$ (coprime to 15)
2. Compute powers of a modulo N:

$$7^1 \bmod 15 = 7 \tag{15}$$
$$7^2 \bmod 15 = 4 \tag{16}$$
$$7^3 \bmod 15 = 13 \tag{17}$$
$$7^4 \bmod 15 = 1 \tag{18}$$
$$\tag{19}$$

3. Quantum part finds order $r = 4$
4. Check: r is even and $7^{r/2} = 7^2 = 4 \neq -1 \pmod{15}$
5. Compute:

$$\gcd(7^{r/2} - 1, 15) = \gcd(4 - 1, 15) = \gcd(3, 15) = 3 \tag{20}$$
$$\gcd(7^{r/2} + 1, 15) = \gcd(4 + 1, 15) = \gcd(5, 15) = 5 \tag{21}$$

6. Factors of 15 are 3 and 5

Complexity and Resource Requirements

- To factor an n-bit number N:
  - Requires $2n$ qubits for input register
  - Requires n qubits for function evaluation
  - Circuit depth: $O(n^3)$
  - Total gate complexity: $O(n^3)$
- Current hardware limitations:
  - State-of-the-art quantum computers: ≈ 100 noisy qubits
  - To factor RSA-2048: Need $\approx 4{,}000$ logical qubits
  - With error correction: Millions of physical qubits
- Conclusion: Large-scale factoring is still years away

Practical Implementations

- Largest number factored with Shor's algorithm: 21 (2012)
- Largest RSA number factored classically: RSA-250 (795 bits, 2020)
- Implementation challenges:
  - Efficient modular exponentiation circuits
  - Error correction for large-scale computations
  - Efficient mapping to physical quantum hardware
- Simplified demonstrations:
  - "Compiled" circuits for specific numbers
  - Textbook vs. practical implementation differences
  - Trade-offs between qubit count and circuit depth

Implications for Cryptography

- Shor's algorithm threatens:
  - RSA encryption
  - Diffie-Hellman key exchange
  - Elliptic Curve Cryptography
- Post-quantum cryptography:
  - Lattice-based cryptography
  - Hash-based cryptography
  - Code-based cryptography
  - Multivariate polynomial cryptography
  - Isogeny-based cryptography
- NIST standardization process underway
- Quantum-resistant algorithms expected to replace current standards

Key Takeaways

- Shor's algorithm shows exponential quantum speedup for factoring
- Combines quantum order finding with classical number theory
- Quantum Fourier Transform enables efficient order finding
- Most significant threat to current public-key cryptography
- Drives development of post-quantum cryptography standards
- Motivates quantum computing research and development
- Demonstrates the potential power of quantum algorithms

Summary of Workshop

We covered key concepts in quantum computing:

- **Module 1**: Quantum fundamentals (qubits, gates, entanglement)
- **Module 2**: Quantum teleportation protocol
- **Module 3**: Deutsch's algorithm (first quantum algorithm)
- **Module 4**: Shor's algorithm (period finding and factorization)

Next steps:

- Implementing these concepts in Qiskit
- Exploring other quantum algorithms (Grover's, VQE, QAOA)
- Investigating quantum error correction
- Keeping up with advances in quantum hardware